

Data-Level Security and Encryption

Lecture 3

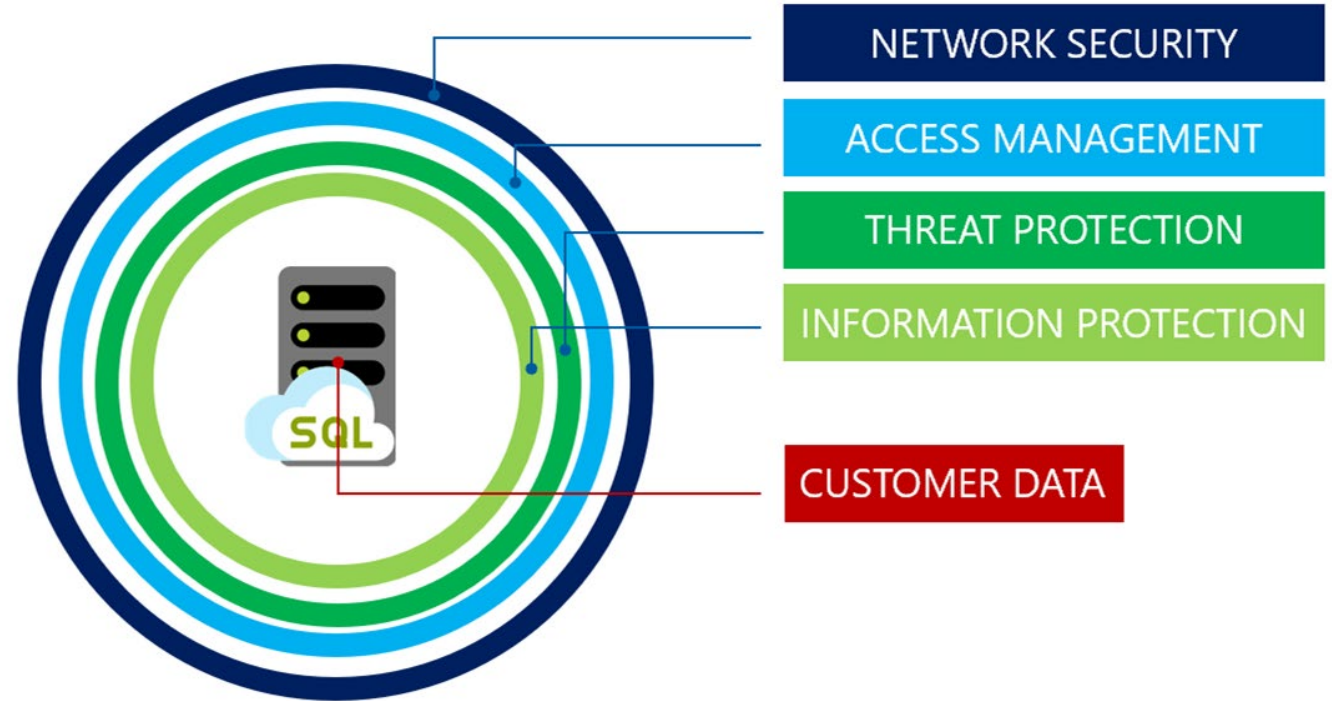
Data-Level Security

Part 1

Topics

- ▶ Schemas
- ▶ Ownership Chaining
- ▶ Impersonation
- ▶ Row-Level Security
 - ▶ Security Predicates
 - ▶ Security Policies
 - ▶ Implementing Row-Level Security
- ▶ Dynamic Data Masking

Below the principal hierarchy, SQL Server provides a rich set of functionality for securing data.

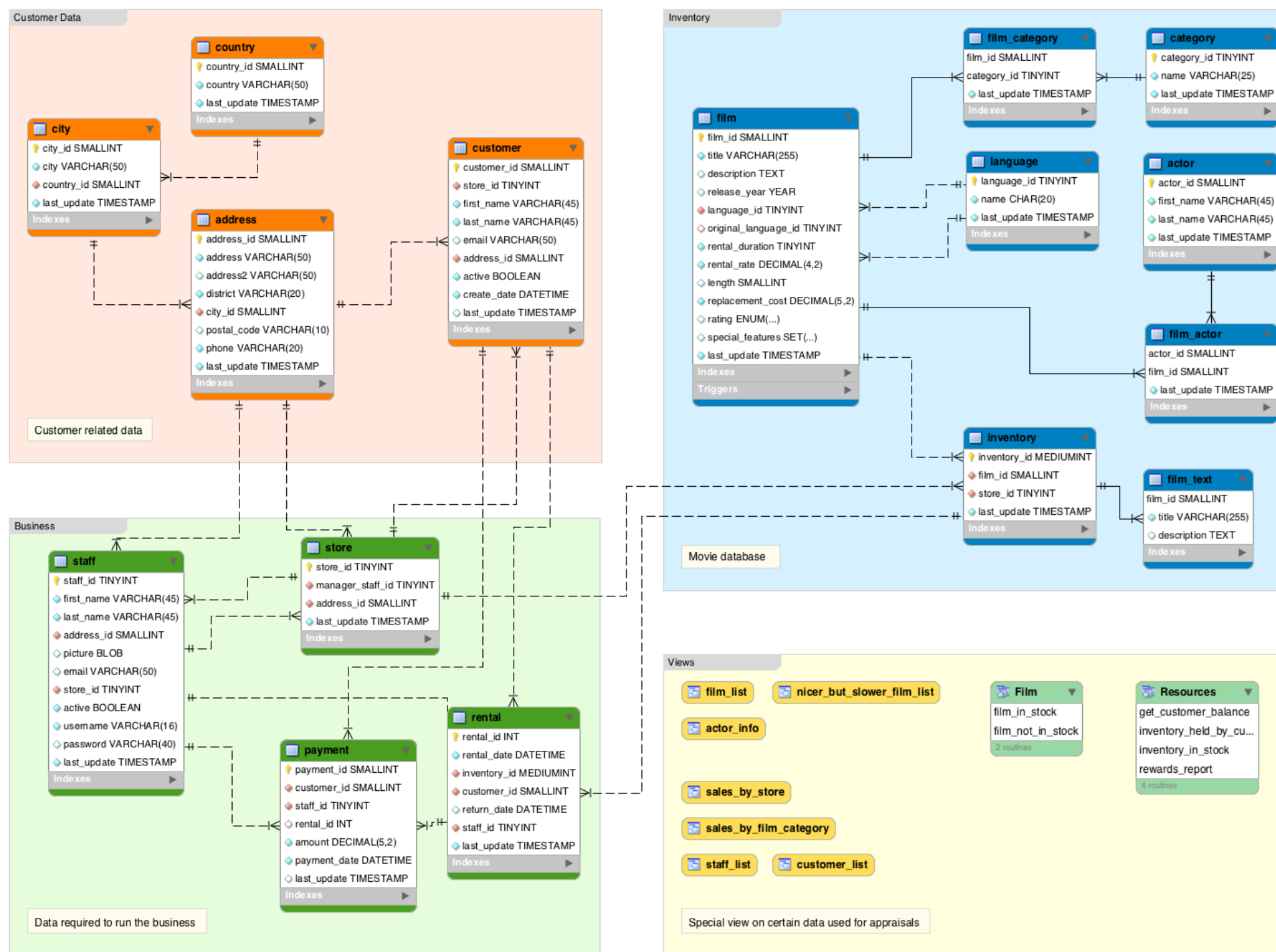




Schemas

- ▶ A database schema is considered the “blueprint” of a database which describes how the data may relate to other tables or other data models.
- ▶ Schemas provide a logical namespace for database objects and provide a layer of abstraction between objects and their owners.
- ▶ Well-defined schemas simplify the management of permissions, grouping tables by business rules as opposed to technical joins.

Example DB Schemas





Schemas

- ▶ Listing 4-1 demonstrates how to create a schema called Chapter4 in the WideWorldImporters database. It then assigns the user Danni, SELECT permission to the schema.
- ▶ To change a table's schema post-creation, use the ALTER SCHEMA TRANSFER statement, as demonstrated in Listing 4-2. This script creates a table without specifying a schema. This means that it is automatically placed in the dbo schema. It is then moved to the Chapter4 schema.

```
USE WideWorldImporters
GO
CREATE SCHEMA Chapter4 ;
GO
GRANT SELECT ON SCHEMA::Chapter4 TO Danni ;
GO
```

Listing 4-1 Create a New Schema

```
USE WideWorldImporters
GO
CREATE TABLE ChangeSchema
(
    ID int
) ;
GO
ALTER SCHEMA Chapter4 TRANSFER
dbo.ChangeSchema ;
GO
```

Listing 4-2 Transfer an Object to a Different Schema



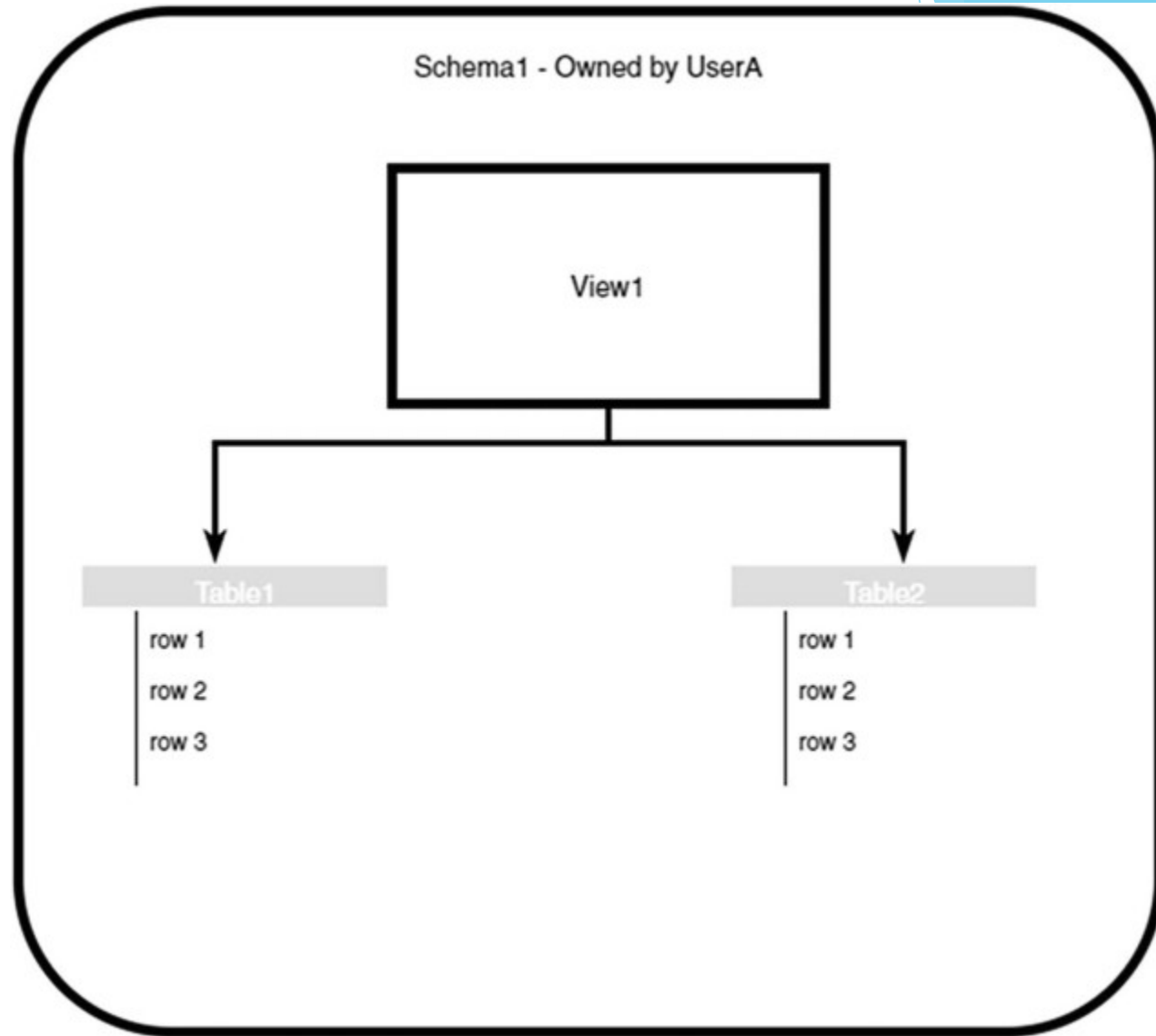
Ownership Chaining

enables you to use group membership or execution context to control access to rows in a database table

- ▶ In previous versions of SQL Server, row-level security could be rather tricky to implement.
- ▶ The standard way to implement row-level security was to use views or procedures that limited the amount of data that was returned.
- ▶ Users can be granted permissions to the procedures and views, which form an abstraction layer, **without granting the user permissions to the underlying tables.**
- ▶ This method works because of a concept called ownership chaining.
- ▶ When multiple objects are called sequentially by a query, SQL Server regards them as a chain.
- ▶ When chaining objects together, the permissions are evaluated differently, depending on what principal owns the schema(s) in which the objects reside.

Ownership Chaining

- ▶ If grant UserB the SELECT permissions on specific rows within Table1, then can create a view that stores a query that returns the rows that this user is permitted to see.
- ▶ At this point, the user can run a SELECT statement from the view, as opposed to the base table.
- ▶ If the user has SELECT permission on the view and the view shares an owner with the base table(s), then the permissions on the underlying table are not evaluated, and the query succeeds.



Permissions check
against view
and user has
permissions, so
query succeeds

Figure 4-2 Successful ownership chain

Ownership Chaining

- ▶ The ownership chain is broken in the event that one of the objects the view is based on does not have the same owner as the view.
- ▶ In this scenario, permissions on the underlying table are checked by SQL Server, and an error is returned if the user does not have appropriate permissions to the underlying table.
- ▶ It is important to note that ownership chains lead to DENY assignments being bypassed. This is because neither the GRANT nor DENY assignments of the user will be evaluated.

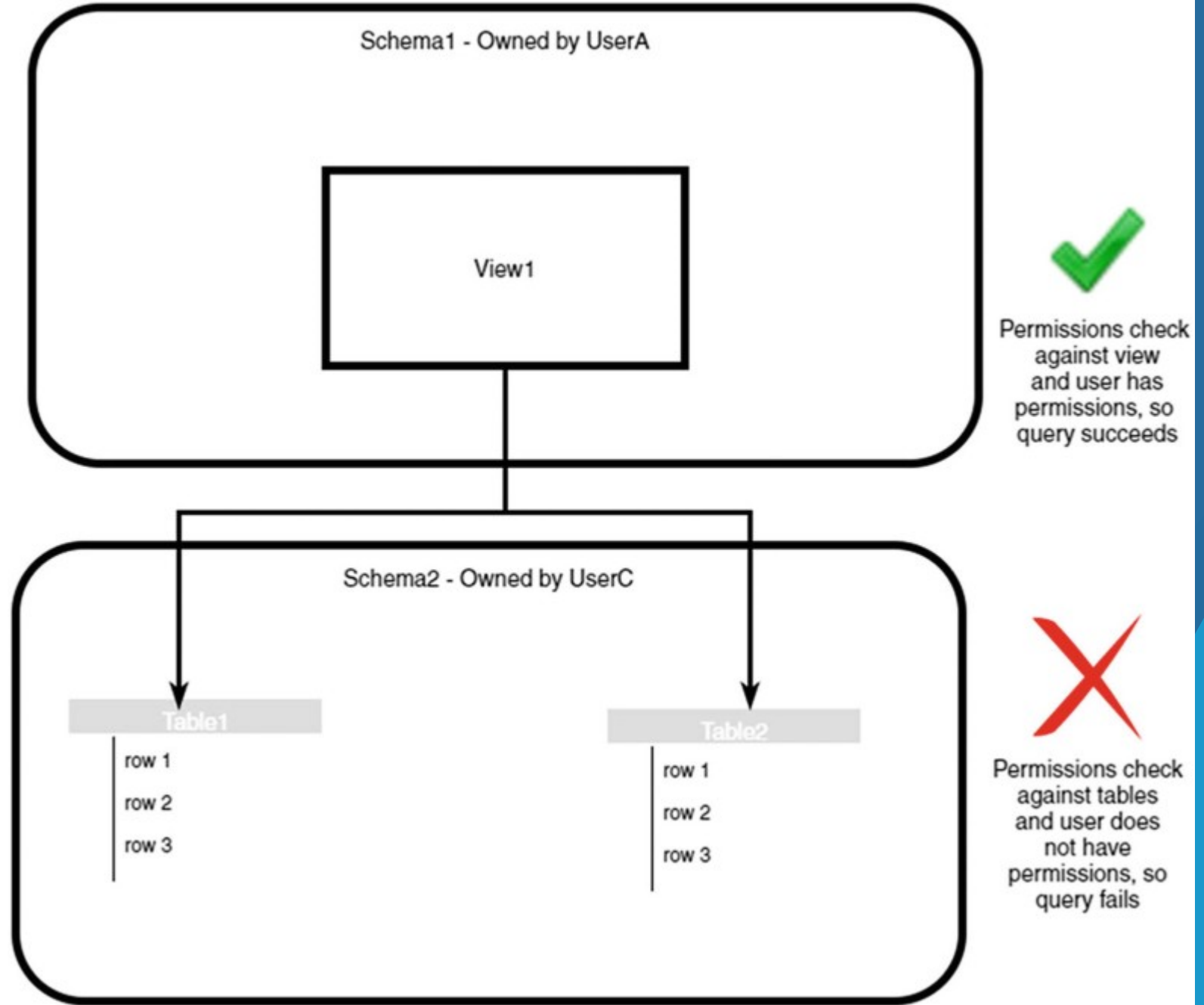


Figure 4-3 Broken ownership chain



Impersonation

- ▶ Impersonation refers to the practice of executing T-SQL statements or code modules under the context of a different security principal.
- ▶ This helps you to enforce the principal of least privilege by assigning fewer permissions to users but elevating those permissions at the point when a section of code is executed.
- ▶ In SQL Server, impersonation can be implemented through the EXECUTE AS clause.
- ▶ The EXECUTE AS clause can be placed in the header of a stored procedure, function, or DML Trigger.
- ▶ EXECUTE AS can also be used during a session to change the security context.
- ▶ Table 4-1 details the context specifications that can be specified when using EXECUTE AS.
- ▶ Table 4-2 explains the usage of each of the context specifications.



Impersonation

Usage	Context Specification
Session	• LOGIN • USER
Procedures, Functions, and DML Triggers	• CALLER • SELF • OWNER • USER
Database-level DDL Triggers	• CALLER • SELF • USER
Server-level DDL Triggers	• CALLER • SELF • LOGIN
Queues	• CALLER • SELF • USER

Table 4-1 EXECUTE AS Context Specifications

Context Specification	Description
CALLER	The code will execute under the original context. This is the default behavior for all modules, except queues.
SELF	The code will execute under the context of the principal that created, or last altered, the module.
OWNER	The code will execute under the context of the principal that owns the module or the schema in which the module resides.
USER	The code will execute under the context of a specific database user.
LOGIN	The code will run under the context of a specific Login.

Table 4-2 Context Specification Usage



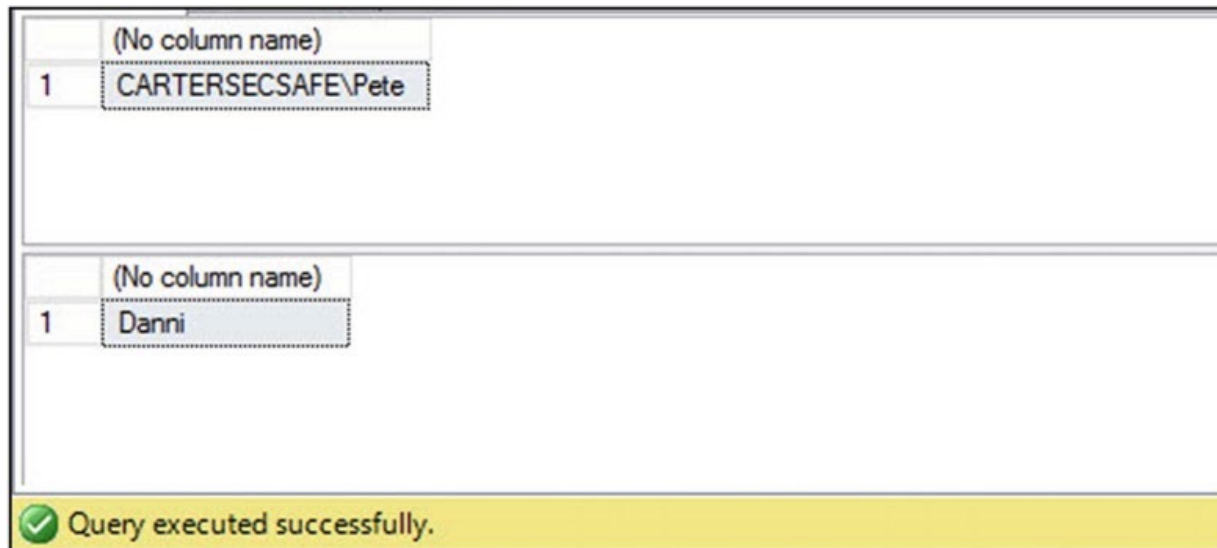
Impersonation

- ▶ The script in Listing 4-3 demonstrates the EXECUTE AS functionality by using a system function named SUSER_SNAME().
- ▶ This function returns the name of a Login from a SID that is passed a parameter.
- ▶ If no parameter is passed, then it will return the name of the Login of the current security context.
- ▶ The results of this query are shown in Figure 4-4. As you can see, the first query ran under the context of my Login. After using the EXECUTE AS statement, however, the security context changed to Danni.
- ▶ To revert back to the original security context, the code must use the REVERT statement.
- ▶ If no REVERT statement is supplied in the code, then the code will continue to run under the modified context until the end of the Session, or code module.
- ▶ The user that creates the code module that contains the EXECUTE AS clause or the user executing the ad-hoc SQL within a session, using the EXECUTE AS clause, **must have the IMPERSONATE permission** on the security context that the code will run under.

Impersonation

```
--Execute under current security context  
SELECT SUSER_SNAME() ;  
--Switch to the context of Danni  
EXECUTE AS USER = 'Danni' ;  
--Execute under Danni's security context  
SELECT SUSER_NAME() ;
```

Listing 4-3 Change Security Context



The screenshot displays two query result grids. The first grid shows the result of the initial query, and the second grid shows the result after switching to the 'Danni' security context. A status bar at the bottom indicates the query was executed successfully.

	(No column name)
1	CARTERSECSAFE\Pete

	(No column name)
1	Danni

✓ Query executed successfully.

Figure 4-4 Results of change security context query



Row-Level Security

- ▶ allows DBAs to simplify the management of fine-grain security by providing an out-of-the-box technology.
- ▶ security would be implemented in the middle tier and the application would connect to SQL Server using a single Login
- ▶ can also assist with improving architectural principals by pushing logic and security to the back end.
- ▶ implemented through a Security Policy and Security Predicates.



Row-Level Security - Security Predicates

- ▶ A Security Predicate is a function that is applied to a result set to determine what rows can be returned, or modified, by the user accessing the data.
- ▶ The functions are inline table-valued functions, which must be created by the DBA.
- ▶ There are two types of Security Predicate that can be implemented:
 - A) Filter Predicates filter the rows that are returned to a user when they query a table or view.
 - B) Block Predicates will return an error if they are violated. This type of predicate can be used to explicitly block INSERT, UPDATE, and DELETE statements, which would violate the predicate.



Row-Level Security - Security Policies

- ▶ A Security Policy binds the Security Predicate(s) to tables and views.
- ▶ It is the Security Policy that invokes the Security Predicate and specifies how the predicate should be used
- ▶ A Security Policy can be created using the `CREATE SECURITY POLICY` statement.
- ▶ Table 4-3 details the arguments that can be specified when creating a Security Policy.



Row-Level Security - Security Policies

Argument	Description
<code>schema_name.security_policy_name</code>	The name to be assigned to the Security Policy and the schema in which it should be created
<code>ADD</code>	Specifies if the predicate should be <code>FILTER</code> or <code>BLOCK</code>
<code>PREDICATE</code>	The two-part name of the Security Predicate
<code>column_name expression</code>	The column name or expression that should be used as the input parameter for the Security Predicate
<code>table_schema.table_name</code>	The two-part name of the target table to which the Security Policy will be applied
<code>block_DML_operations</code>	If the <code>ADD</code> argument set to <code>BLOCK</code> , then the DML operations to block will be defined.
<code>STATE</code>	Specifies if the Security Policy will be enabled on creation
<code>SCHEMABINDING</code>	Specifies if Security Predicates that are bound to the Security Policy must be created with <code>SCHEMABINDING</code>
<code>NOT FOR REPLICATE</code>	Specifies that the security policy should not be executed when a replication agent modifies the target object

Table 4-3 CREATE SECURITY POLICY Arguments



Implementing RLS

- ▶ To show the implementation of RLS, use this scenario of having managers be able to view only details of employees who report to them.
- ▶ To implement RLS, first create a Security Predicate and a Security Policy.
- ▶ The Security Predicate defines which rows are available by a user based on the manager column of the table.
- ▶ To show the example, create the employee table such as in Listing 4-4.
- ▶ The employee hierarchy uses the HIERARCHYID data type, which is a complex data type first implemented in SQL Server 2008. For more information on HIERARCHYID data type, refer to <https://docs.microsoft.com/en-us/sql/t-sql/data-types/hierarchyid-data-type-method-reference?view=sql-server-ver15>

Row-Level Security - Implementing RLS

```
USE WideWorldImporters
GO
CREATE TABLE Application.Employees
(
    EmployeeID      INT          NOT NULL      PRIMARY KEY,
    NINumber        NVARCHAR(15) NOT NULL,
    LoginName       NVARCHAR(256) NOT NULL,
    Manager         HIERARCHYID  NULL,
    Title           NVARCHAR(256) NOT NULL
) ;
INSERT INTO Application.Employees
VALUES(1, 295847284, 'WideWorldImporters\ken0', '/', 'Chief Executive Officer'),
(2 ,245797967 , 'WideWorldImporters\terri0', '/1/', 'Vice President of Engineering'),
(3 ,509647174 , 'WideWorldImporters\roberto0', '/1/1/', 'Engineering Manager'),
(4 ,112457891 , 'WideWorldImporters\rob0',      '/1/2/', 'Senior Tool Designer'),
(5 ,695256908 , 'WideWorldImporters\gail0', '/1/1/1/', 'Design Engineer'),
(6 ,998320692 , 'WideWorldImporters\jossef0', '/1/1/2/', 'Design Engineer'),
(7 ,134969118 , 'WideWorldImporters\dylan0', '/1/3/', 'Research and Development Manager'),
(8 ,811994146 , 'WideWorldImporters\diane1', '/1/3/1/', 'Research and Development Engineer'),
(9 ,658797903 , 'WideWorldImporters\gigi0', '/1/3/2/', 'Research and Development Engineer'),
(10 ,879342154 , 'WideWorldImporters\michael6', '/1/3/3/', 'Research and Development Manager'),
(12 ,480168528 , 'WideWorldImporters\thierry0', '/1/2/1/', 'Tool Designer'),
(13 ,486228782 , 'WideWorldImporters\janice0', '/1/2/2/', 'Tool Designer'),
(14 ,42487730 , 'WideWorldImporters\michael8', '/1/2/1/', 'Senior Design Engineer'),
(15 ,56920285 , 'WideWorldImporters\sharon0', '/1/2/1/1/', 'Design Engineer'),
(17 ,253022876 , 'WideWorldImporters\kevin0', '/2/', 'Head of Marketing'),
(18 ,222969461 , 'WideWorldImporters\john5', '/2/1/', 'Marketing Specialist'),
(19 ,52541318 , 'WideWorldImporters\mary2', '/2/1/1/', 'Marketing Assistant'),
(20 ,323403273 , 'WideWorldImporters\wanida0', '/2/1/2/', 'Marketing Assistant'),
(21 ,243322160 , 'WideWorldImporters\terry0', '/2/2/', 'Marketing Specialist'),
(22 ,95958330 , 'WideWorldImporters\sariya0', '/2/3/', 'Marketing Specialist'),
(23 ,767955365 , 'WideWorldImporters\mary0', '/2/4/', 'Marketing Specialist'),
(24 ,72636981 , 'WideWorldImporters\jill0', '/2/5/', 'Marketing Specialist'),
(26 ,277173473 , 'WideWorldImporters\peter0', '/3/', 'Production Control Manager') ;
```

Listing 4-4 Create An Application.Employees Table

Row-Level Security - Implementing RLS

- ▶ Listing 4-5 shows how to write a predicate function.
- ▶ The script also creates a new schema called Security where the predicate function will reside.
- ▶ Listing 4-6 creates the Security Policy.

```
USE WideWorldImporters
GO
CREATE SCHEMA Security ;
GO
CREATE FUNCTION Security.fn_securitypredicate(@Manager
HIERARCHYID)
    RETURNS TABLE
WITH SCHEMABINDING
AS
    RETURN SELECT 1 AS fn_securitypredicate_result
    FROM Application.Employees e1
    WHERE @Manager.IsDescendantOf(Manager) = 1
        AND LoginName = 'WideWorldImporters\' + USER_NAME()
;
GO
```

Listing 4-5 Create a Security Predicate

```
CREATE SECURITY POLICY Security.EmployeeSecurityPolicy
ADD FILTER PREDICATE
Security.fn_securitypredicate(Manager) ON
Application.Employees
WITH (STATE=ON, SCHEMABINDING=ON) ;
```

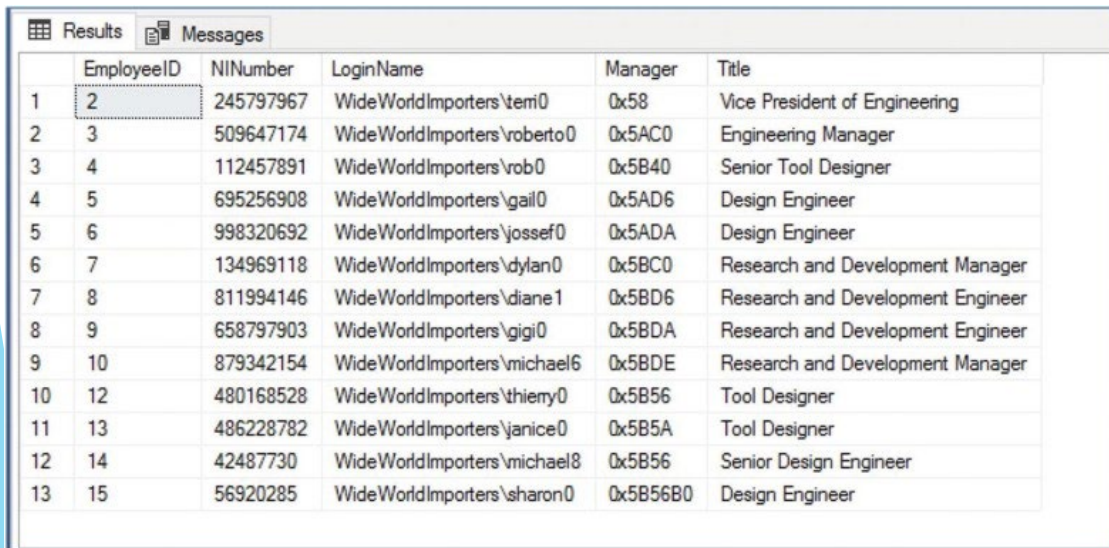
Listing 4-6 Create a Security Policy

Row-Level Security - Implementing RLS

- ▶ Listing 4-8 shows how to test a result with a user terri0 with db_datareader role.
- ▶ The results are displayed in Figure 4-5.

```
USE WideWorldImporters
GO
CREATE USER terri0 WITHOUT LOGIN ;
ALTER ROLE db_datareader ADD MEMBER terri0 ;
GO
EXECUTE AS USER = 'terri0'
    SELECT * FROM Application.Employees ;
REVERT
```

Listing 4-7 Test the RLS

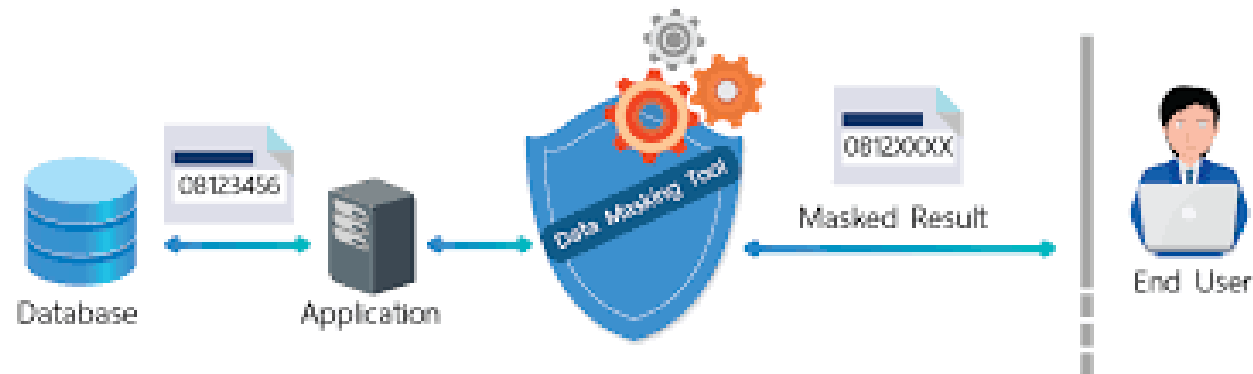


	EmployeeID	NINumber	LoginName	Manager	Title
1	2	245797967	WideWorldImporters\terri0	0x58	Vice President of Engineering
2	3	509647174	WideWorldImporters\roberto0	0x5AC0	Engineering Manager
3	4	112457891	WideWorldImporters\rob0	0x5B40	Senior Tool Designer
4	5	695256908	WideWorldImporters\gail0	0x5AD6	Design Engineer
5	6	998320692	WideWorldImporters\jossef0	0x5ADA	Design Engineer
6	7	134969118	WideWorldImporters\dylan0	0x5BC0	Research and Development Manager
7	8	811994146	WideWorldImporters\diane1	0x5BD6	Research and Development Engineer
8	9	658797903	WideWorldImporters\gigi0	0x5BDA	Research and Development Engineer
9	10	879342154	WideWorldImporters\michael6	0x5BDE	Research and Development Manager
10	12	480168528	WideWorldImporters\thierry0	0x5B56	Tool Designer
11	13	486228782	WideWorldImporters\janice0	0x5B5A	Tool Designer
12	14	42487730	WideWorldImporters\michael8	0x5B56	Senior Design Engineer
13	15	56920285	WideWorldImporters\sharon0	0x5B56B0	Design Engineer

Figure 4-5 RLS test results

Dynamic Data Masking

- ▶ Introduced in SQL Server 2016 which allows non -privileged users to see only a subset of an atomic value in a cell
- ▶ Masks important information through obfuscation
- ▶ Helps improve application architecture by pushing code from middle tier to backend.
- ▶ Table 4-4 shows dynamic masking functions available





Dynamic Data Masking

Function	Supported Data Types	Description
default	<code>char, nchar, varchar, nvarchar, text, ntext, bigint, bit, decimal, int, money, numeric, smallint, smallmoney, tinyint, float, real, date, datetime2, datetime, datetimeoffset, smalldatetime, time, binary, varbinary, image</code>	Fully masks a value. The type of masking depends on the data type of the value
partial	<code>char, nchar, varchar, nvarchar</code>	Accepts a prefix, a masking value, and a suffix
email	<code>char, nchar, varchar, nvarchar</code>	Reveals only the first letter of the e-mail address, the @ symbol, and the domain suffix
random	<code>bigint, decimal, int, numeric, smallint, smallmoney, tinyint, float, real</code>	Replaces a value with a random value, from within a specified range

Table 4-4 Dynamic Data-Masking Functions

Dynamic Data Masking

- ▶ Listing 4-8 shows how to create and populate the Application.CreditCards table while Listing 4-9 adds the mask.
- ▶ Users will only see the last four digits.

```
USE WideWorldImporters
GO
ALTER TABLE Application.CreditCards
ALTER COLUMN CardNumber ADD MASKED WITH
(FUNCTION = 'partial(0,"XXXX-XXXX-XXXX-
",4)');
```

Listing 4-9 Add a Data Mask

```
USE WideWorldImporters
GO
CREATE TABLE Application.CreditCards
(
    CardID          INT          NOT NULL,
    CardType        NVARCHAR(50) NOT NULL,
    CardNumber       NVARCHAR(20) NOT NULL,
    ExpMonth        INT NOT NULL,
    ExpYear         INT NOT NULL,
    CustomerID      INT NOT NULL
) ;
INSERT INTO Application.CreditCards
VALUES(1, 'SuperiorCard', '33332664695310', 10, 20, 991),
(2, 'Distinguish', '55552127249722', 11, 21, 156),
(3, 'ColonialVoice', '77778344838353', 10, 21, 1),
(4, 'ColonialVoice', '77774915718248', 12, 22, 920),
(5, 'Vista', '11114404600042', 12, 22, 949),
(6, 'Distinguish', '55557132036181', 12, 22, 912),
(7, 'Distinguish', '55553635401028', 10, 19, 65),
(8, 'SuperiorCard', '33336081193101', 10, 19, 69),
(9, 'Distinguish', '55553465625901', 12, 19, 846),
(10, 'SuperiorCard', '33332126386493', 12, 19, 495),
(11, 'SuperiorCard', '33335352517363', 11, 19, 6),
(12, 'SuperiorCard', '33334316194519', 11, 18, 79),
(13, 'Vista', '11119775847802', 10, 18, 60),
(14, 'Distinguish', '55553287727410', 12, 18, 60),
(15, 'SuperiorCard', '33336866065599', 11, 20, 817),
(16, 'Vista', '11111985451507', 10, 22, 22),
(17, 'ColonialVoice', '77771220960729', 12, 22, 929),
(18, 'ColonialVoice', '77773971683137', 10, 20, 473),
(19, 'ColonialVoice', '77779803886862', 20, 19, 505),
(20, 'SuperiorCard', '33332150058339', 10, 21, 436) ;
```

Listing 4-8 Create And Populate The Application.CreditCards Table

Dynamic Data Masking

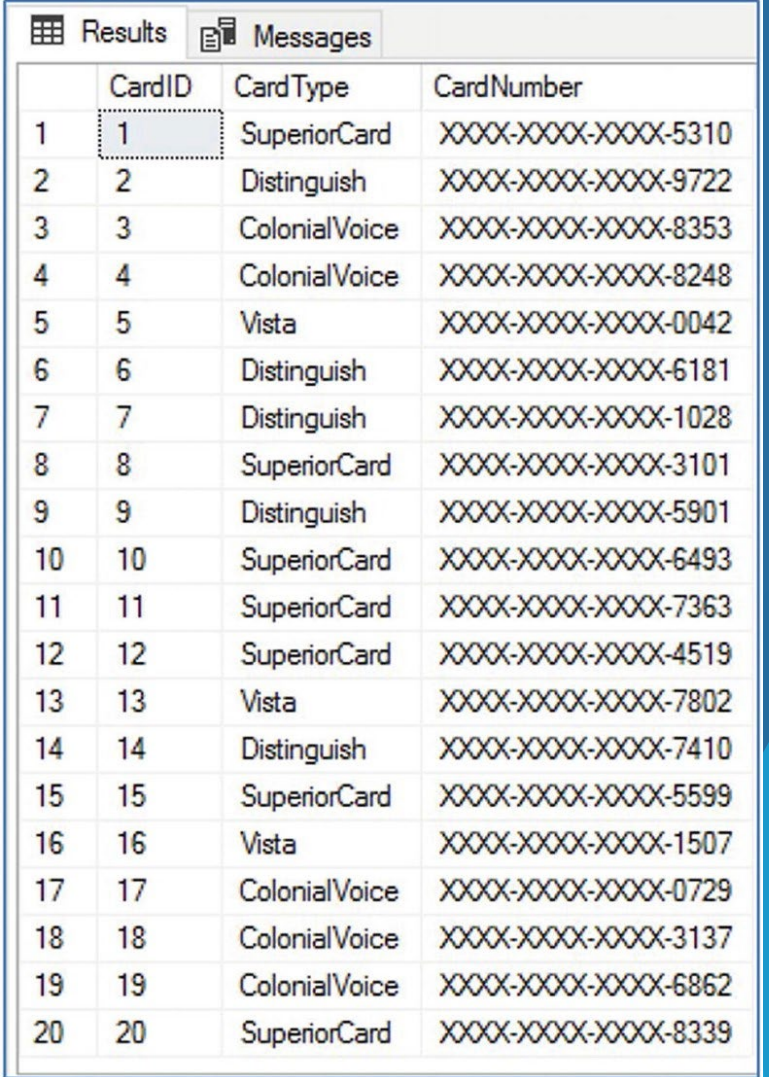
- ▶ To test the mask, create a user terri0 with SELECT privileges to the Application.CreditCards table and run the query in Listing 4-10.
- ▶ The result is Figure 4-6.
- ▶ To reveal the full value, UNMASK permission must be granted such as in listing 4-11. Now try the query again.

```
EXECUTE AS USER = 'terri0' ;  
SELECT  
    CardID  
    ,CardType  
    ,CardNumber  
FROM Application.CreditCards ;  
REVERT
```

Listing 4-10 Query the Sales.CreditCard Table

```
GRANT UNMASK TO terri0 ;
```

Listing 4-11 Grant the UNMASK Permission



The screenshot shows a SQL Server query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with four columns: 'CardID', 'CardType', and 'CardNumber'. The 'CardNumber' column contains masked values represented by 'X's. The table lists 20 rows of data.

	CardID	CardType	CardNumber
1	1	SuperiorCard	XXXX-XXXX-XXXX-5310
2	2	Distinguish	XXXX-XXXX-XXXX-9722
3	3	ColonialVoice	XXXX-XXXX-XXXX-8353
4	4	ColonialVoice	XXXX-XXXX-XXXX-8248
5	5	Vista	XXXX-XXXX-XXXX-0042
6	6	Distinguish	XXXX-XXXX-XXXX-6181
7	7	Distinguish	XXXX-XXXX-XXXX-1028
8	8	SuperiorCard	XXXX-XXXX-XXXX-3101
9	9	Distinguish	XXXX-XXXX-XXXX-5901
10	10	SuperiorCard	XXXX-XXXX-XXXX-6493
11	11	SuperiorCard	XXXX-XXXX-XXXX-7363
12	12	SuperiorCard	XXXX-XXXX-XXXX-4519
13	13	Vista	XXXX-XXXX-XXXX-7802
14	14	Distinguish	XXXX-XXXX-XXXX-7410
15	15	SuperiorCard	XXXX-XXXX-XXXX-5599
16	16	Vista	XXXX-XXXX-XXXX-1507
17	17	ColonialVoice	XXXX-XXXX-XXXX-0729
18	18	ColonialVoice	XXXX-XXXX-XXXX-3137
19	19	ColonialVoice	XXXX-XXXX-XXXX-6862
20	20	SuperiorCard	XXXX-XXXX-XXXX-8339

Figure 4-6 Results of masked query



Summary

- ▶ SQL Server provides a rich suite of functionality in managing data-level security
- ▶ Schemas provide a namespace for objects by allowing permissions assignment based on roles
- ▶ Row Level Security (RLS) allows restricting rows within a table based on user security attributes
- ▶ Dynamic data masking allows non-privileged users to see a partially obfuscated value.

Encryption in SQL

Part 2

Topics

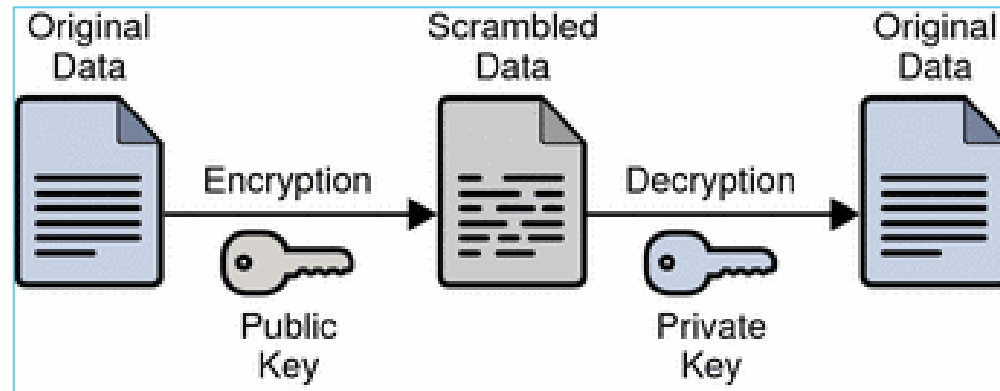
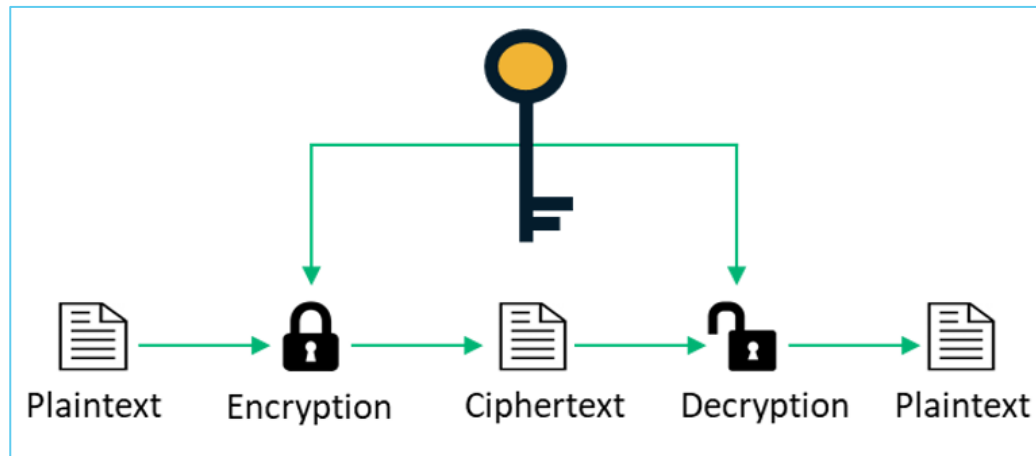
- ▶ General Encryption Concepts
 - ▶ Defense in Depth
 - ▶ Symmetric and Asymmetric Keys
 - ▶ Certificates and Self-Signed Certificates
 - ▶ Windows Data Protection API
- ▶ SQL Server Encryption Concepts
 - ▶ Master Keys
 - ▶ EKM and Key Stores
 - ▶ SQL Server Encryption Hierarchy
- ▶ Encrypting Data
 - ▶ Encrypting with Password/Passphrase
 - ▶ Encrypting with Keys and Certificates
- ▶ Transparent Data Encryption (TDE)
 - ▶ Considerations for TDE with Other Technologies
 - ▶ Implementing and Administering TDE
 - ▶ Backup the Certificate
 - ▶ Migrating an Encrypted Database
- ▶ Always Encrypted
 - ▶ Implementing Always Encrypted
 - ▶ Limitations



Introduction

- ▶ Encryption is a process of obfuscating data with the use of an algorithm that uses keys and certificates.
- ▶ Even if data is accessed or stolen by attackers, then it will be useless unless they also obtain the keys that were used to encrypt it.
- ▶ Adds an additional layer of security over and above access control, but it does not replace the requirement for an access control strategy.
- ▶ Has the potential to degrade performance and increase the size of data, so use discernment

General Encryption Concepts



Symmetric Keys

- Same key used to both encrypt/decrypt data
- Faster than asymmetric keys
- TDE uses Triple-DES

Asymmetric Keys

- Two keys - one public one private.
- One key is used for decryption while the other for encryption.
- Both keys are mathematically linked
- Encryption key is the public key.
Decryption key is the private key.
- SQL Server uses RSA 512-2048



General Encryption Concepts

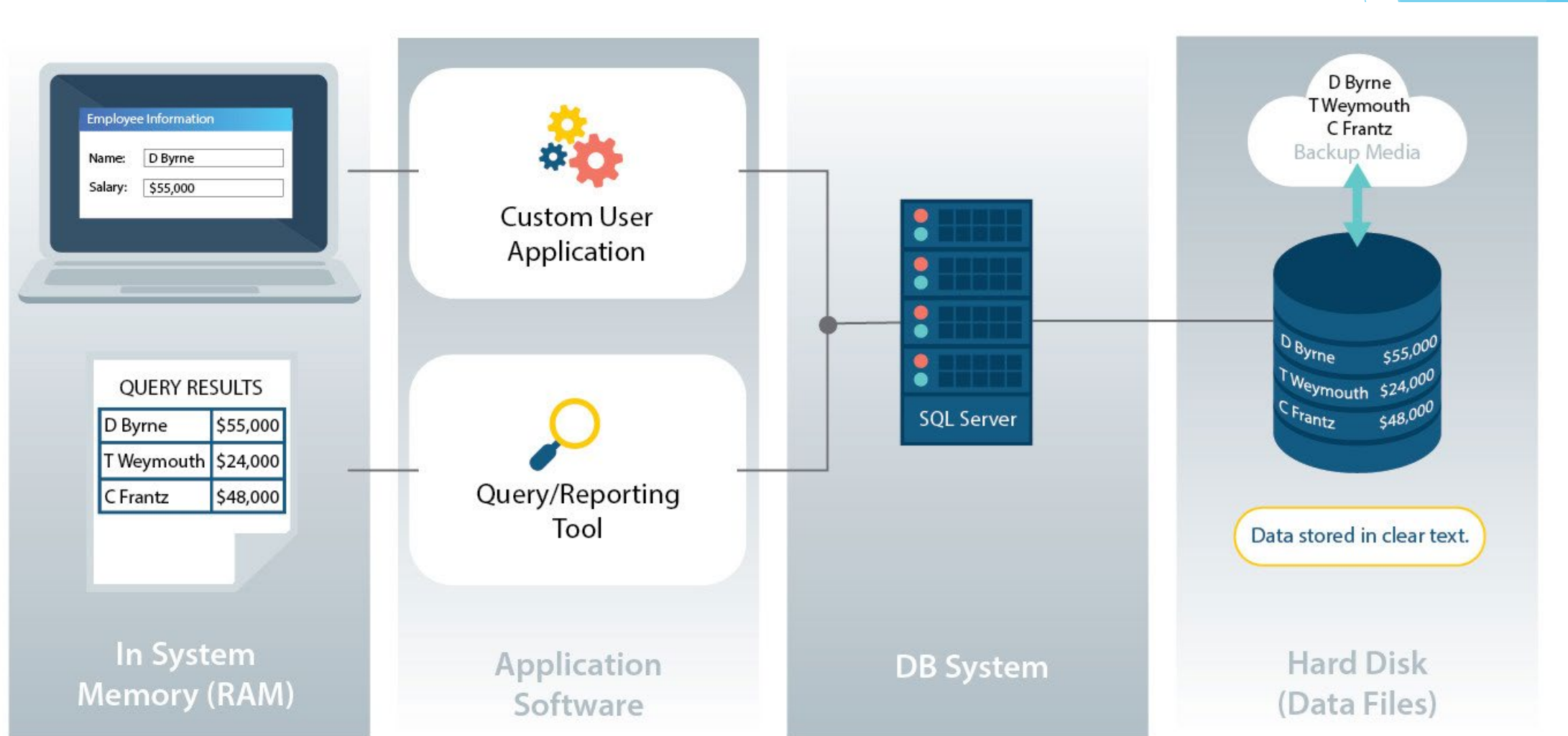
Certificates

- ▶ X509 format
- ▶ Issued by a trusted source known as a certificate authority.
- ▶ Uses asymmetric key cryptography but additionally provides a digital signature statement binding the public key to a principle/device
- ▶ A self-signed cert is one that is signed by the same entity it certifies. Can be created by SQL Server.

Windows Data Protection API

- ▶ A cryptographic application programming interface with WIN OS.
- ▶ Allows keys to be encrypted using user secret information or domain secret information
- ▶ DPAPI is used to encrypt the Service Master Key at the top level of SQL Server encryption hierarchy.

SQL Server Encryption Concepts



SQL Server Encryption Concepts

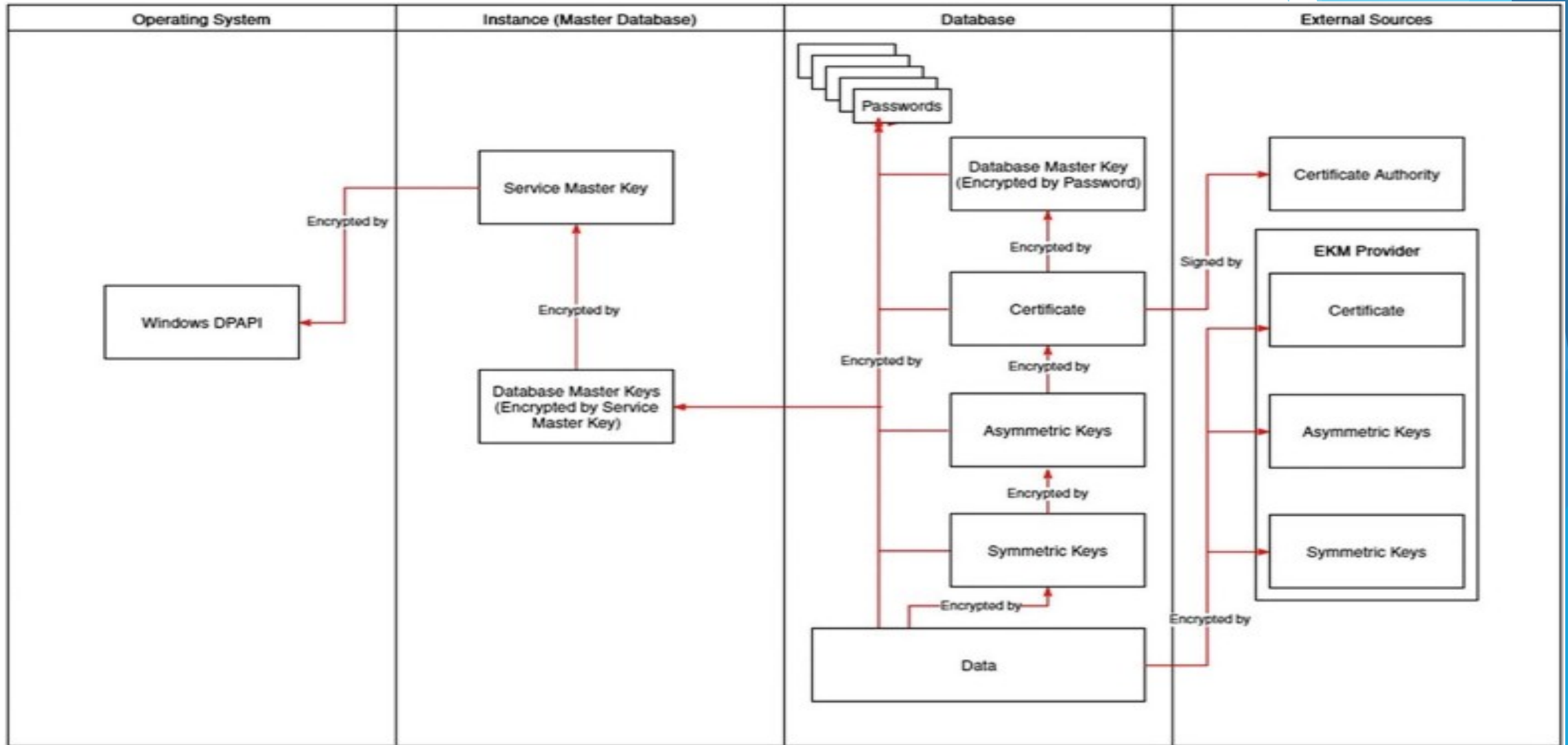


Figure 5-1 SQL Server encryptions hierarchy



SQL Server Encryption Concepts

- ▶ Relies on hierarchy of keys and certificates
- ▶ Root is the Service Master Key, this is automatically created when the instance is built.
- ▶ Used to encrypt database master keys, credentials and passwords for linked servers using DPAPI.
- ▶ Stored in the Master database. Always one key per instance.
- ▶ From SQL Server 2012, Service Master Key is AES256 key, but older versions use Triple-DES.
- ▶ Good practice to regenerate the key if upgrading an instance from 2008R2 or below.
- ▶ To regenerate a key, all keys and certificates within the instance's encryption hierarchy must be decrypted then re-encrypted. This is resource intensive and to be done during maintenance only.
- ▶ Listing 5-1 shows how to regenerate the Service Master Key
- ▶ If the process fails to decrypt and reencrypt, then the whole regeneration process fails. This can be changed using the FORCE keyword, but it will leave any data that cannot be decrypted and re-encrypted unusable.

```
ALTER SERVICE MASTER KEY REGENERATE ;
```

Listing 5-1 Regenerate the Service Master Key



SQL Server Encryption Concepts

- ▶ Also important to backup the Service Master key.
- ▶ Backup in a secure, offsite location for availability in DR scenarios, as well as migration scenarios.
- ▶ Listing 5-2 shows how to backup and restore the Service Master Key. If the restore is identical, SQL Server lets you know the data does not need to be decrypted and reencrypted.
- ▶ If your instance does not use encryption features, then backup of the key is not required.
- ▶ The service_master_key is the name of the key file as opposed to a folder. By convention, there is no extension.

```
--Backup Service Master Key
BACKUP SERVICE MASTER KEY
TO FILE = 'c:\keys\service_master_key'
ENCRYPTION BY PASSWORD = 'Pa$$w0rd' ;
--Restore Service Master Key
RESTORE SERVICE MASTER KEY
FROM FILE =
'c:\keys\service_master_key'
DECRYPTION BY PASSWORD = 'Pa$$w0rd' ;
```

Listing 5-2 Backup and Restore the Service Master Key

SQL Server Encryption Concepts

- ▶ Database Master Key is a key for AES256.
- ▶ Used to encrypt private keys and certificates within a database.
- ▶ It is encrypted using a password as the secret, but a copy is created which is encrypted using the Service Master Key.
- ▶ This allows the Database Master Key to be opened automatically when required. If no copy exists, then has to be opened manually.
- ▶ Likewise, important to backup Database Master Key as losing it would result in data loss below the encryption hierarchy.
- ▶ Listing 5-3 shows how to create and backup, then attempt to restore it.
- ▶ Similarly can use FORCE keyword, with possibility of data loss in similar scenario.

```
USE WideWorldImporters
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD =
'Pa$$w0rd' ;
BACKUP MASTER KEY TO FILE =
'c:\keys\Chapter5_master_key'
ENCRYPTION BY PASSWORD = 'Pa$$w0rd';
RESTORE MASTER KEY
FROM FILE = 'c:\keys\Chapter5_master_key'
DECRYPTION BY PASSWORD = 'Pa$$w0rd' --The
password in the backup file
ENCRYPTION BY PASSWORD = 'Pa$$w0rd' ; --The
password it will be encrypted within the
database
```

Listing 5-3 Administering a Database Master Key



SQL Server Encryption Concepts

- ▶ EKM module allows generating and managing keys and certificates used to secure SQL Server Data within a 3rd party HSM
- ▶ Provides interface with SQL Server using MSCAPI
- ▶ Key is not stored with the data
- ▶ Also benefit from advanced features from 3rd party vendors such as key rotation and secure key disposal
- ▶ HSM is also hardware-based, so improved performance.
- ▶ Key stores provide secure storage and a trusted source for keys and certificates. Use Windows Certificate Store for Windows Server.
- ▶ Other options: Azure Key Vault, Amazon Key Management Services, Keywhiz, Vault.



Encrypting Data

- ▶ Encrypting data can be done using either:
 - a) Password
 - b) Encryption hierarchy



Encrypting Data with Password

- ▶ `ENCRYPTBYPASSPHRASE()` function - allows encryption of data directly using a password or passphrase
- ▶ Reuse the credit card table in last section. This table stores credit card details in plaintext, so is insecure and against regulatory requirements.
- ▶ To encrypt the credit card number column:
 - 1) If dynamic masking is deployed, use Listing 5-4's script to remove it first.
 - 2) Add a new column to the table to hold the encrypted data
 - 3) Populate the new column using `ENCRYPTBYPASSPHRASE()` such as with Listing 5-6. Parameters for the function are in Table 5-1.
 - 4) Set the column to not allow NULL values and drop the original column as in Listing 5-7. Whoever, do not drop this if you plan to continue with further examples. This script is for demonstration only.

Encrypting Data with Password

```
USE WideWorldImporters
GO
ALTER TABLE Application.CreditCards
ALTER COLUMN CardNumber DROP MASKED ;
```

Listing 5-4 Drop Dynamic Data Mask

```
USE WideWorldImporters
GO
ALTER TABLE Application.CreditCards ADD
    CardNumberEncrypted varbinary(8000) NULL ;
```

Listing 5-5 Add a New Column to Hold the Encrypted Credit Card Numbers

```
UPDATE Application.CreditCards
SET CardNumberEncrypted = ENCRYPTBYPASSPHRASE('Pa$$w0rd',
CardNumber, 0) ;
```

Listing 5-6 Populate the Encrypted Column

```
--Set CardNumberEncrypted column to be
NOT NULL
ALTER TABLE Application.CreditCards
ALTER COLUMN CardNumberEncrypted
VARBINARY(256) NOT NULL ;
--Do not run following section, if you
plan to follow later examples
ALTER TABLE Application.CreditCards
    DROP COLUMN CardNumber ;
```

Listing 5-7 Set Encrypted Column NOT NULL and Drop Original Column

Parameter	Description
Passphrase	The password or phrase that will be used to generate a symmetric key
Cleartext	The value to be encrypted
add_authenticator	Specifies if an authenticator should be used
authenticator	The value to be used to derive an authenticator

Table 5-1 ENCRYPTBYPASSPHRASE() Parameters



Encrypting Data with Password

- ▶ To decrypt, use the DECRYPTBYPASSPHRASE() function. Parameters are in Table 5-2.
- ▶ To write a procedure for decryption, refer to Listing 5-9. Note that as well as decryption, the result must be converted back to NVARCHAR value for results to be meaningful.
- ▶ An additional security measure is to apply the procedure such as Listing 5-10 for users to only view their own credit card number.

Parameter	Description
passphrase	The password or phrase that will be used to decrypt the data
ciphertext	The value to be decrypted
add_authenticator	Specifies if an authenticator will be required to decrypt the data
authenticator	The authenticator data

Table 5-2 DECRYPTBYPASSPHRASE() Parameters

Encrypting Data with Password

```
CREATE PROCEDURE ReturnCredCardInfo @CustomerID
INT
AS
BEGIN
    SELECT
        Cust.CustomerID
        , Cust.CustomerName
        , Cust.PrimaryContact
        , Cards.CardNumber
        , Cards.CardType
        , Cards.ExpMonth
        , Cards.ExpYear
    FROM Website.Customers Cust
    INNER JOIN Application.CreditCards Cards
        ON Cust.CustomerID = Cards.CustomerID
    WHERE Cust.CustomerID = @CustomerID ;
END
```

Listing 5-8 Return Credit Card Information

```
CREATE PROCEDURE ReturnCredCardInfo @CustomerID
INT
AS
BEGIN
    SELECT
        Cust.CustomerID
        , Cust.CustomerName
        , Cust.PrimaryContact
        ,
        CONVERT(NVARCHAR(25), DECRYPTBYPASSPHRASE('Pa$$w
Ord', Cards.CardNumberEncrypted, 0)) AS
        CardNumber
        , Cards.CardType
        , Cards.ExpMonth
        , Cards.ExpYear
    FROM Website.Customers Cust
    INNER JOIN Application.CreditCards Cards
        ON Cust.CustomerID = Cards.CustomerID
    WHERE Cust.CustomerID = @CustomerID ;
END
```

Listing 5-9 Modify the Procedure to Work with the Encrypted Column

```

USE WideWorldImporters
GO
CREATE PROCEDURE dbo.AddCreditCard
    @CustomerID          INT
    , @CreditCardNumber  NVARCHAR(25)
    , @CardType           NVARCHAR(50)
    , @ExpMonth           TINYINT
    , @ExpYear            SMALLINT
    , @Password           NVARCHAR(128)
AS
BEGIN
    DECLARE @CreditCardID INT ;
    INSERT INTO Application.CreditCards (
        CardType ,
        ExpMonth ,
        ExpYear ,
        CardNumberEncrypted
    )
    VALUES (
        @CardType,
        @ExpMonth,
        @ExpYear,
        ENCRYPTBYPASSPHRASE(@Password, @CreditCardNumber, 0)
    ) ;
END
GO
CREATE PROCEDURE ReturnCredCardInfo
    @CustomerID          INT
    , @Password           NVARCHAR(128)
AS
BEGIN
    SELECT
        CONVERT(NVARCHAR(25), DECRYPTBYPASSPHRASE(@Password, Cards.CardNumberEncrypted, 0)) AS CreditCardNumber
    FROM Application.CreditCards Cards
    WHERE Cards.CustomerID = @CustomerID ;
END
GO

```

Listing 5-10 Encrypt and Decrypt Data, Based Upon a User's Password



Encrypting Data with Keys and Certificates

- ▶ If using the encryption hierarchy, data can be encrypted using symmetric keys, asymmetric keys or certificates.
- ▶ Table 5-3 shows the different cryptographic functions for each.
- ▶ Keys and certificates within the hierarchy can be further encrypted using keys and certificates up the line.
- ▶ For performance reasons, always use symmetric keys unless there is a very good reason not to.

Encryption Type	Encryption Function	Decryption Function
Symmetric	ENCRYPTBYKEY ()	DECRYPTBYKEY ()
Asymmetric	ENCRYPTBYASYKEY ()	DECRYPTBYASYKEY ()
Certificate	ENCRYPTBYCERT ()	DECRYPTBYCERT ()

Table 5-3 Cryptographic Functions

Encrypting Data with Keys and Certificates

- ▶ CREATE CERTIFICATE SQL arguments are detailed in Table 5-4. It can also be used to import a certificate or create one that uses existing keys stored in a file.
- ▶ CREATE SYMMETRIC KEY arguments are detailed in Table 5-5.

Argument	Description
AUTHORIZATION	Specifies the owner of the certificate
ACTIVE FOR BEGIN_DIALOG	Specifies if the certificate can be used to initiate a Service Broker conversation
ENCRYPTION BY PASSWORD	Specifies the password that will be used to encrypt the certificate's private key
WITH SUBJECT	Specifies a subject for the certificate
START_DATE	Specifies a date on which the certificate becomes valid
EXPIRY_DATE	Specifies a date on which the certificate expires, after which it will no longer be valid

Table 5-4 CREATE CERTIFICATE Arguments

Argument	Description
AUTHORIZATION	Specifies the owner of the key
FROM PROVIDER	If the key is managed by an EKM provider, specifies the EKM provider to use
KEY_SOURCE	Specifies a passphrase from which to generate the key
IDENTITY_VALUE	Specifies a value from which to generate a GUID that can be used for temporary tagging data that will be encrypted with a temporary key
PROVIDER_KEY_NAME	Specifies the name by which the key is known to the EKM provider, if an EKM provider is to be used
CREATION_DISPOSITION	If an EKM provider will be used, specifies if a new key should be created in the EKM or if an existing key should be used. Acceptable values are: • CREATE_NEW - Specifies that a new key will be created in the EKM provider • OPEN_EXISTING - Specifies that an existing key will be opened in the EKM provider
ENCRYPTION BY	Specifies how the key will be encrypted. Acceptable values are: • CERTIFICATE (followed by the name of the certificate) • PASSWORD (followed by the password to use) • SYMMETRIC KEY (followed by the name of the key to use) • ASYMMETRIC KEY (followed by the name of the key to use)
ALGORITHM	Specifies the algorithm to use to encrypt the key. Acceptable values are: • DES • TRIPLE_DES • TRIPLE_DES_3KEY • RC2 • RC4 • RC4_128 • DESX • AES_128 • AES_192 • AES_256

Table 5-5 CREATE SYMMETRIC KEY Arguments

Encrypting Data with Keys and Certificates

- ▶ ENCRYPTBYKEY function parameters are found in Table 5-6.
- ▶ Listing 5-11 shows how to create a symmetric key and use it to encrypt the CardNumberEncrypted column.
- ▶ Open the key before using and close the key after completion.

Parameter	Description
key_GUID	The GUID of the key that will be used to encrypt the data
cleartext	The value to be encrypted
add_authenticator	Specifies if an authenticator should be used
authenticator	The value to be used to derive an authenticator

Table 5-6 ENCRYPTBYKEY() Parameters

```
USE WideWorldImporters
GO
--Create the certificate
CREATE CERTIFICATE CreditCardCert
WITH SUBJECT = 'Credit Card Numbers';
GO
--Create the symmetric key
CREATE SYMMETRIC KEY CreditCardKey
WITH ALGORITHM = AES_128
ENCRYPTION BY CERTIFICATE CreditCardCert;
--Open the key
OPEN SYMMETRIC KEY CreditCardKey
DECRYPTION BY CERTIFICATE CreditCardCert;
--Encrypt the data, using the symmetric key
UPDATE Application.CreditCards
    SET CardNumberEncrypted =
    ENCRYPTBYKEY(Key_GUID('CreditCardKey'), CardNumber);
--Close the key
CLOSE SYMMETRIC KEY CreditCardKey ;
```

Listing 5-11 Encrypt Data With A Symmetric Key

Encrypting Data with Keys and Certificates

- ▶ Decrypt using the `DECRYPTBYKEY()` function which has parameters in Table 5-7.
- ▶ Listing 5-12 demonstrates how to use `DECRYPTBYKEY()` to read the `CardNumberEncrypted` column.

```
USE WideWorldImporters
GO
--Open the key
OPEN SYMMETRIC KEY CreditCardKey
DECRYPTION BY CERTIFICATE CreditCardCert;
--Decrypt the data, using the symmetric key
SELECT CONVERT(NVARCHAR(30),
DECRYPTBYKEY(CardNumberEncrypted)) AS
CreditCardNumber
FROM Application.CreditCards ;
--Close the key
CLOSE SYMMETRIC KEY CreditCardKey ;
```

Listing 5-12 Decrypt Data With `DECRYPTBYKEY()`

Parameter	Description
Ciphertext	The value to be decrypted
add_authenticator	Specifies if an authenticator will be required to decrypt the data
authenticator	The authenticator data

Table 5-7 `DECRYPTBYKEY()` Parameters



Transparent Data Encryption

- ▶ A feature that encrypts all data pages and log files of a database using a symmetric key called the Database Encryption Key.
- ▶ This key is stored in the boot record in the database and encrypted using a server certificate stored within the Master database.
- ▶ If TDE is enabled, data and log pages are encrypted before they are written to disk and decrypted when they are read into memory. This process is transparent to users. Applications do not need to be modified to access the data.
- ▶ No bloating compared to encryption of data within columns, the database is the same size.
- ▶ Similarly there is performance overhead.



Transparent Data Encryption

► Some considerations:

- 1) Can encrypt In-Memory OLTP, but the data within the In-Memory filegroup is not encrypted even when data is persisted
- 2) Even though the memory optimized data is not encrypted, log records associated with in-memory transactions are encrypted.
- 3) Possible to encrypt databases that use FILESTREAM, but data within a FILESTREAM file group is not encrypted.
- 4) Full text indexes are encrypted for new ones, but existing ones are only encrypted after they are imported during upgrade.
- 5) Full-text indexing is not a good practice because data written to disk is in plaintext during the full-text indexing scan operation.
- 6) If database is replicated, then manually enable TDE on the subscribers
- 7) TempDB is always encrypted using TDE if any user database on the instance has TDE enabled.
- 8) TDE is incompatible with instant file initialization.
- 9) Files used by Buffer Cache Extensions will not be encrypted by TDE. This must be encrypted using system-level encryption tools.

Transparent Data Encryption

Implementing TDE involves the following steps:

- a) Creating a Database Master Key for the `Master` database (If one does not already exist)
- b) Creating a Certificate or Asymmetric key in the `Master` database
- c) Creating a Database Encryption Key in the database that you wish to encrypt
- d) Altering the database to enable Transparent Database Encryption

Listing 5-13 Encrypt the WideWorldImporters Database

```
USE Master
GO
--Create the Database Master Key (if
it does not already exist)
CREATE MASTER KEY ENCRYPTION BY
PASSWORD = 'Pa$$w0rd';
GO
--Create the Server Certificate
CREATE CERTIFICATE TDECert WITH
SUBJECT = 'Certificate For TDE';
GO
USE WideWorldImporters
GO
--Create the Database Encryption Key
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_128
ENCRYPTION BY SERVER CERTIFICATE
TDECert ;
GO
--Enable TDE on the database
ALTER DATABASE WideWorldImporters
SET ENCRYPTION ON ;
GO
```



Transparent Data Encryption

Argument	Description
WITH ALGORITHM	Specifies the algorithm that should be used by the Database Encryption Key. Acceptable values are: • AES_128 • AES_192 • AES_256 • TRIPLE_DES_3KEY
ENCRYPTION BY SERVER	Specifies the certificate or asymmetric key that will be used to encrypt the Database Encryption Key. Acceptable values are: • CERTIFICATE (followed by the name of the certificate to use) • ASYMMETRIC KEY (followed by the name of the Asymmetric Key to use)

Table 5-8 CREATE DATABASE ENCRYPTION KEY Arguments



Transparent Data Encryption

- ▶ Administering TDE include a few scenarios:
 - 1) Backup the certificate - listing 5-14 shows how to do this.
 - 2) Migrating an Encrypted Database - Restoring a certificate is shown in Listing 5-15.

Also make sure SQL Server service account has permissions to the certificate and key files in the OS. If errors are received stating the certificate is not valid, does not exist or do not have permissions to it, check the restore immediately.



Transparent Data Encryption

```
BACKUP CERTIFICATE TDECert  
TO FILE = 'C:\certificates\TDECert'  
WITH PRIVATE KEY  
(file='C:\certificates\TDECertKey',  
ENCRYPTION BY PASSWORD='Pa$$w0rd') ;
```

Listing 5-14 Backing Up the Certificate

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD =  
'Pa$$w0rd' ;  
GO  
CREATE CERTIFICATE TDECert  
FROM FILE = 'C:\Certificates\TDECert'  
WITH PRIVATE KEY  
(  
FILE = 'C:\Certificates\TDECertKey',  
DECRYPTION BY PASSWORD = 'Pa$$w0rd'  
) ;
```

Listing 5-15 Preparing for a Database Migration



Always Encrypted

- ▶ First introduced in SQL Server 2016
- ▶ Protects data against privileged users such as sysadmins
- ▶ Helps with compliance issues for sensitive data when support is outsourced to third-party vendors.
- ▶ Uses two keys: column encryption key and column master key. The column master key can be a key or a certificate located within an external store.
- ▶ With two layered keys, SQL Server only has to store an encrypted value of the column encryption key instead of having it as plaintext. The master key is stored in an external key store. This can be HSM, Windows Certificate Store or an EKM provider.
- ▶ The client driver handles the encryption and decryption data. Supported drivers can be found at : <https://docs.microsoft.com/en-us/sql/relational-databases/security/encryption/always-encrypted-client-development>
- ▶ Figure 5-2 depicts the high-level architecture of Always Encrypted.

Always Encrypted

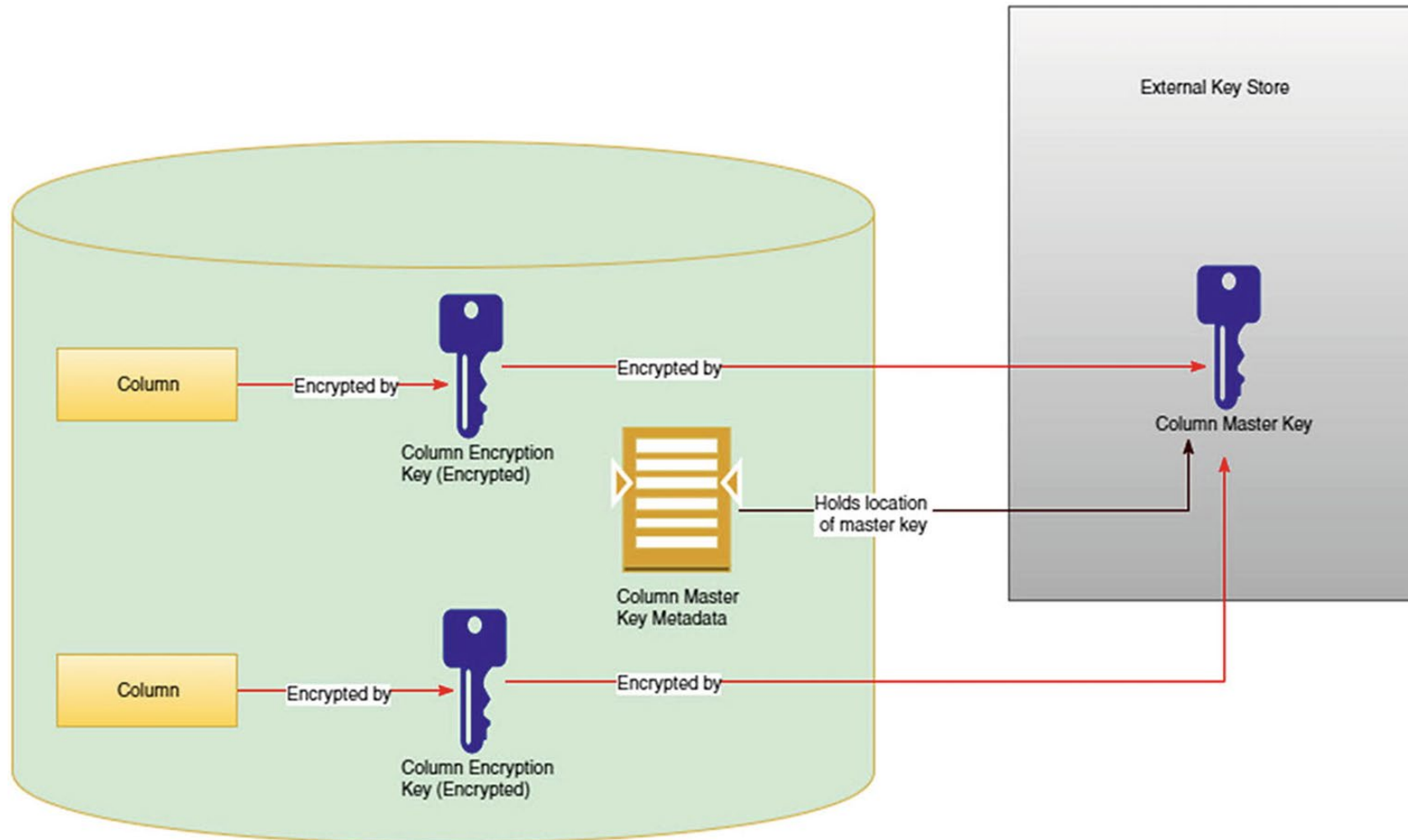


Figure 5-2 Always Encrypted architecture

Always Encrypted

- ▶ To implement Always Encrypted, use either T-SQL, PowerShell or SSMS GUI.
- ▶ In this example, use Always Encrypted to secure the CardType, CreditCardNumber, ExpMonth and ExpYear columns of the Application.CreditCards table.
- ▶ Create a column master key and store it in Windows Certificate Store using the GUI in Figure 5-3.
- ▶ Provide it a name and select the type of store for the Key Store. Table 5-9 shows all possible values.

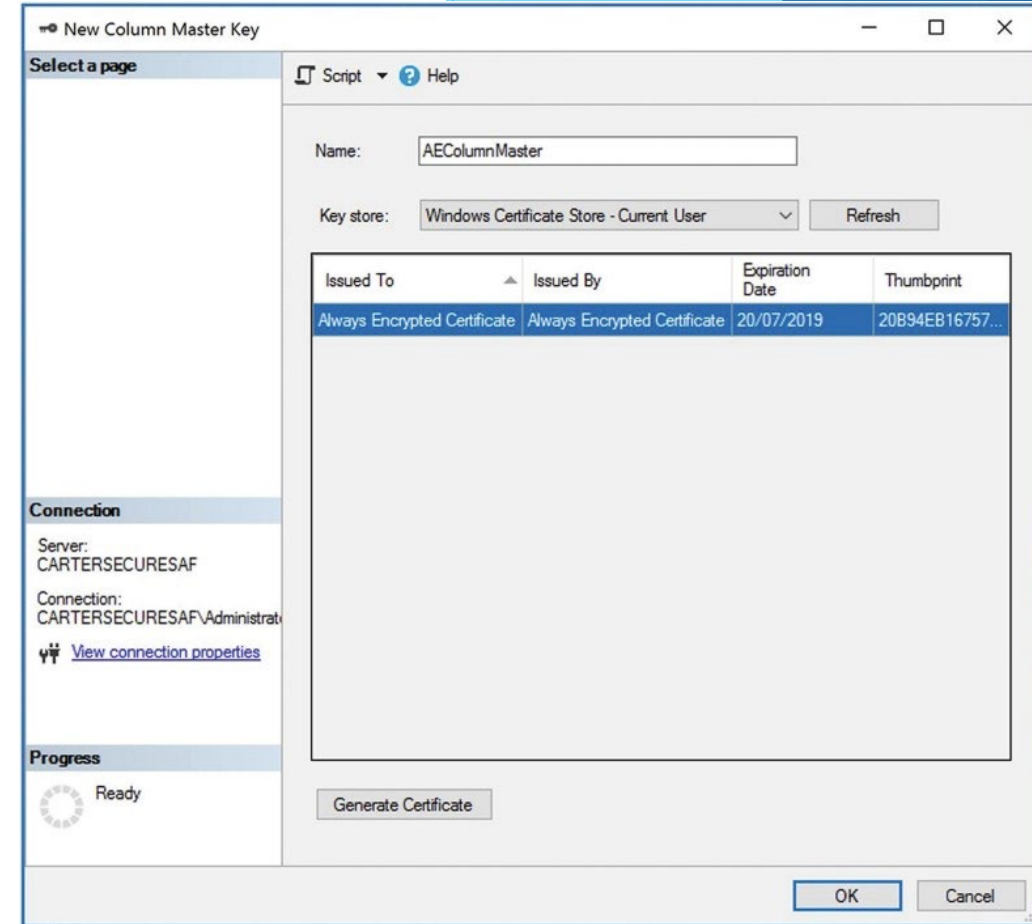


Figure 5-3 New Column Master Key dialog box

Key Store Type	Description
Windows Certificate Store - Current User	The key or certificate is stored in the area of the Windows Certificate Store that is reserved for the profile of the user that created the certificate. This option may be appropriate if you use the database engine's service account interactively to create the certificate.
Windows Certificate Store - Local Machine	The key or certificate is stored in the area of the Windows Certificate Store that is reserved for the local machine.
Azure Key Vault	The key or certificate is stored in the Azure Key Vault EKM service.
Key Storage Provider (CNG)	The key or certificate is stored in an EKM store that supports Cryptography API: Next Generation.

Table 5-9 Key Store Values

Always Encrypted

- ▶ If a certificate is generated, it will immediately appear in the key store as in Figure 5-4.
- ▶ To generate a column encryption key, select New Column Encryption Key under the database's Security->AlwaysEncryptedKeys->ColumnEncryptionKeys node in Object explorer, which will display a GUI as in Figure 5-5.

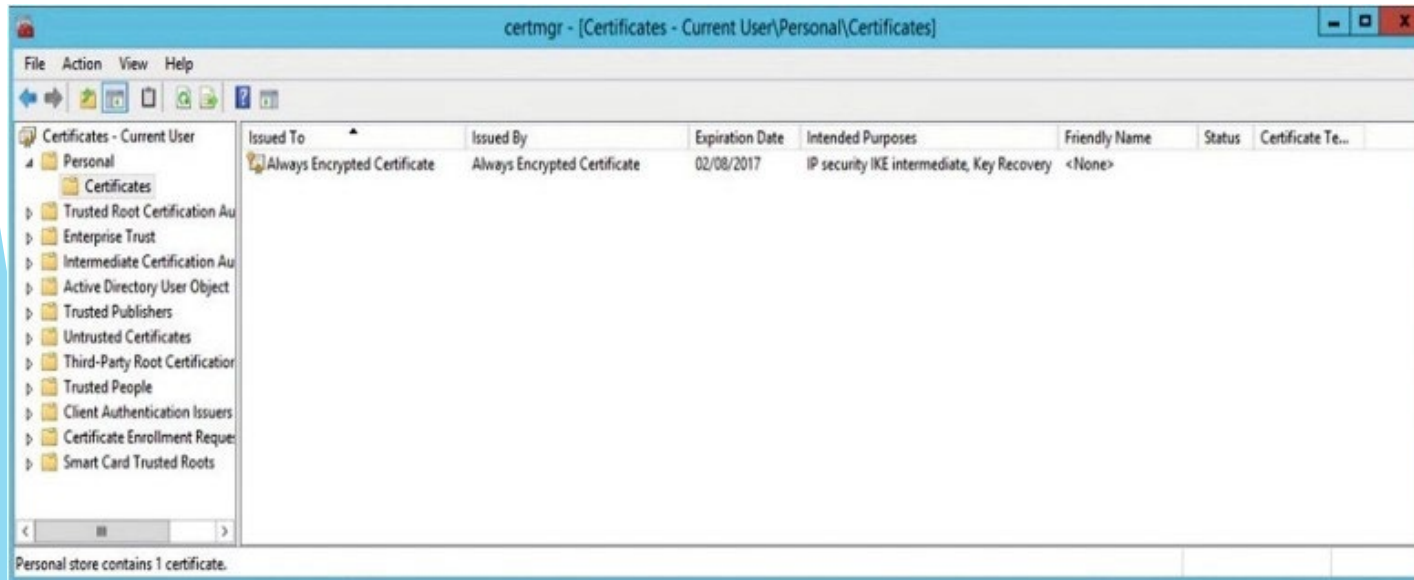


Figure 5-4 Windows Certificate Store

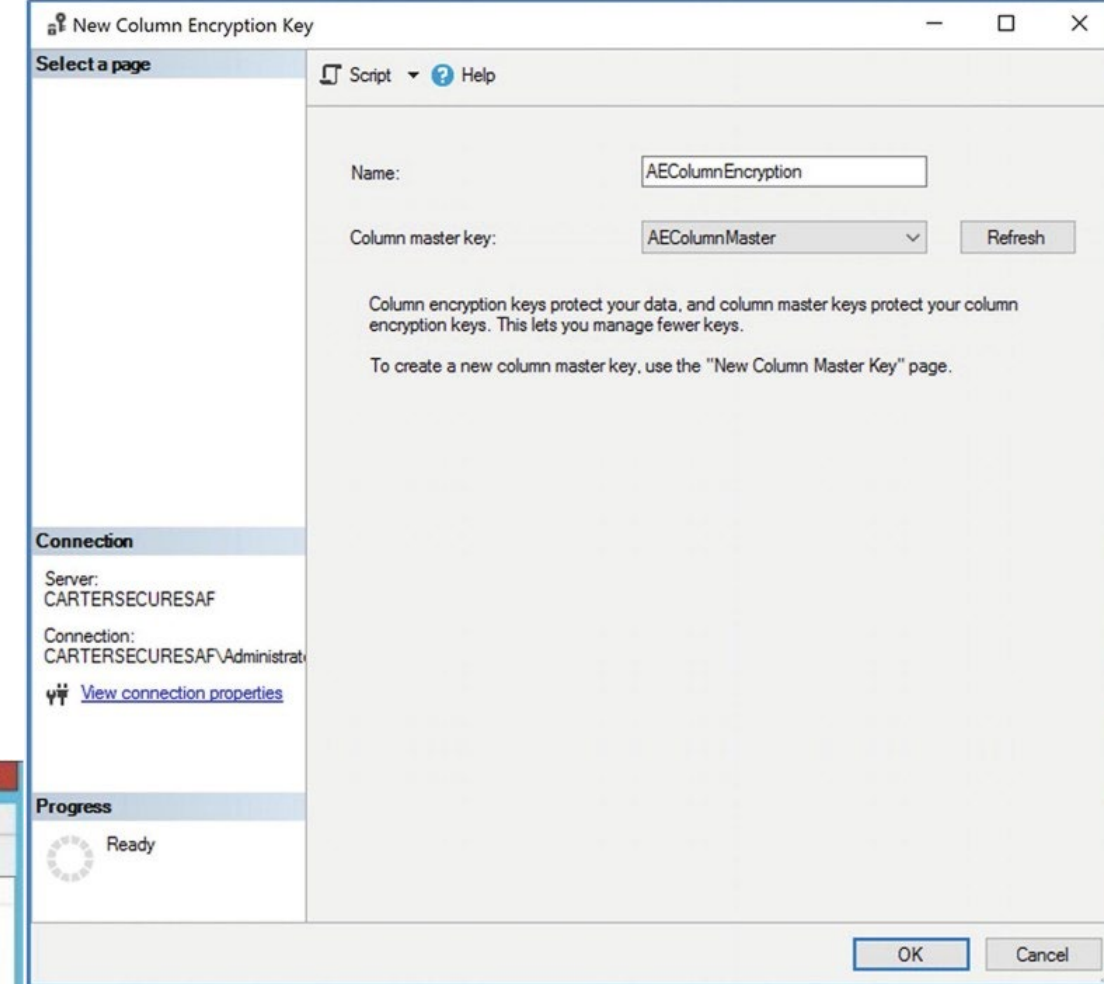


Figure 5-5 New Column Encryption Key dialog box

Always Encrypted

- ▶ The final step is to encrypt the columns. Go to Tables and select EncryptColumns under the CreditCards table. Figure 5-6 is the GUI displayed. Select the columns to be encrypted.
- ▶ Choose either:
 - 1) deterministic - always produce the same encrypted value for similar plaintext.
 - 2) randomized - different encrypted values for similar plaintext
- ▶ Note that only BIN2 is the collation that supports Always Encrypted. Also only encrypt during maintenance windows.
- ▶ The Master Key Configuration page in Figure 5-8 will inform that no further configuration is required. Otherwise, if create new column encryption keys was selected, this page would allow the association with the column master key.
- ▶ The Run Settings page in Figure 5-8 provides the option of performing encryption immediately or scripting in PowerShell, and the Summary page provides the overview of actions to be performed. Encryption is only performed when click Finish.

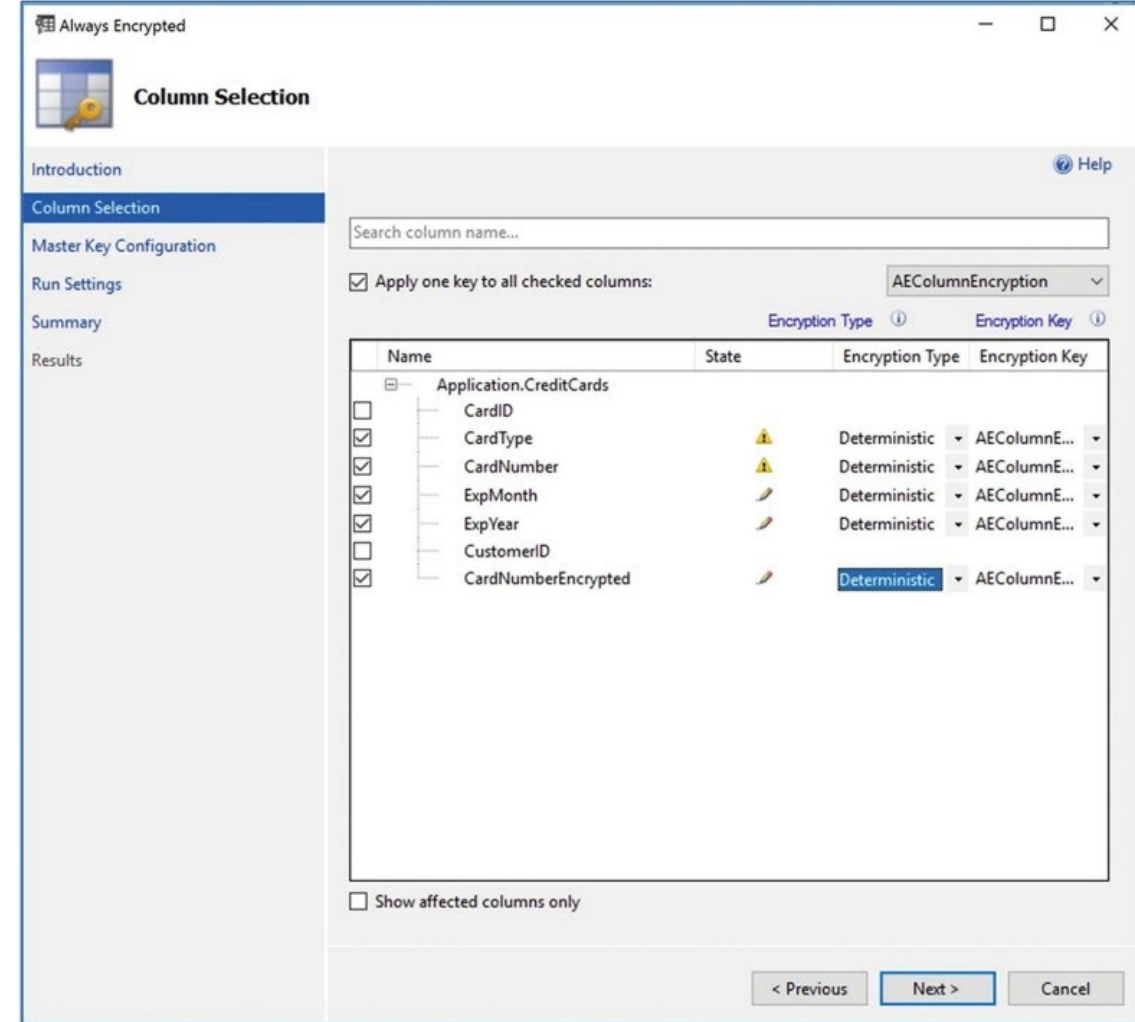


Figure 5-6 Column selection page

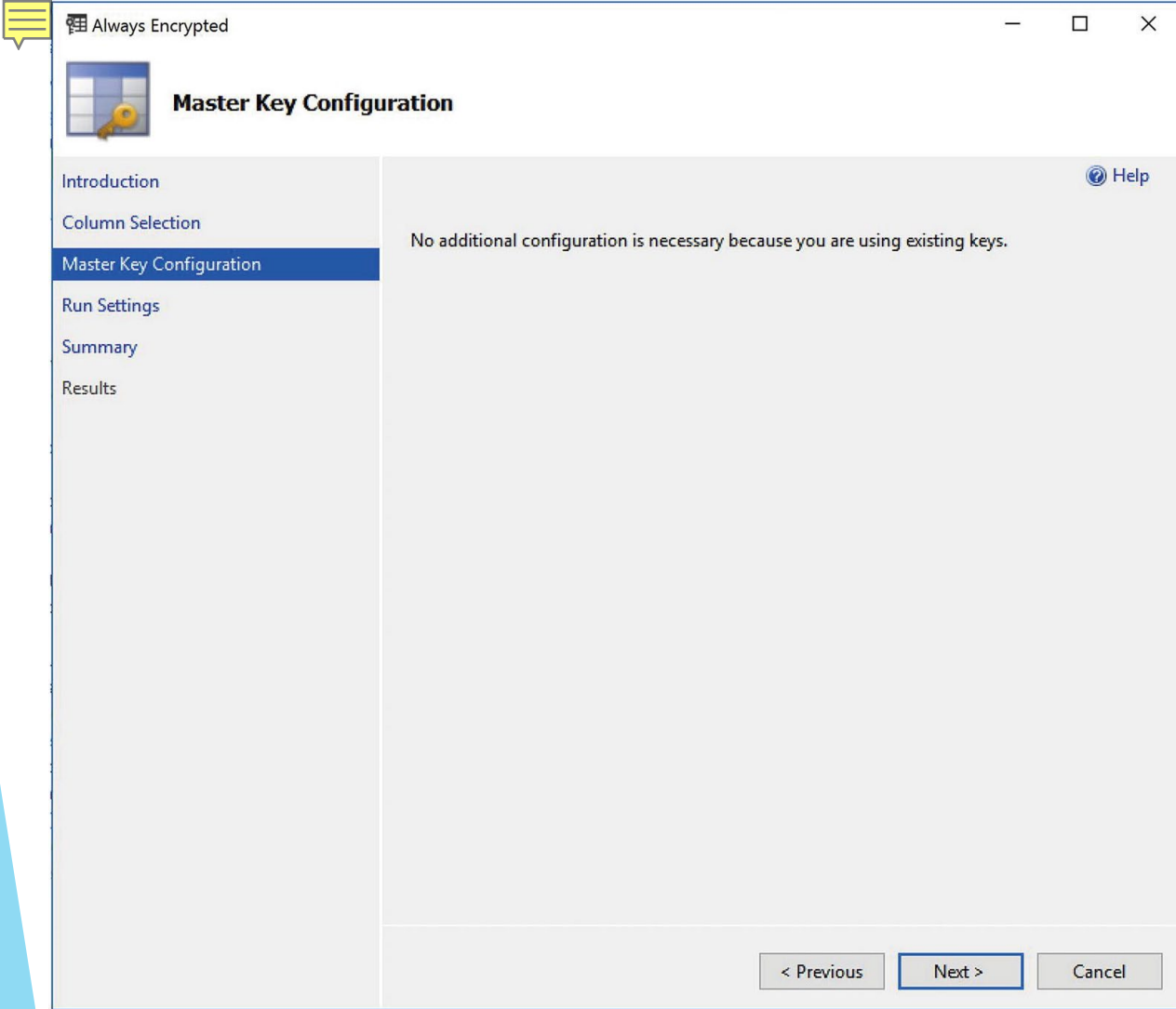


Figure 5-7 Master Key Configuration page

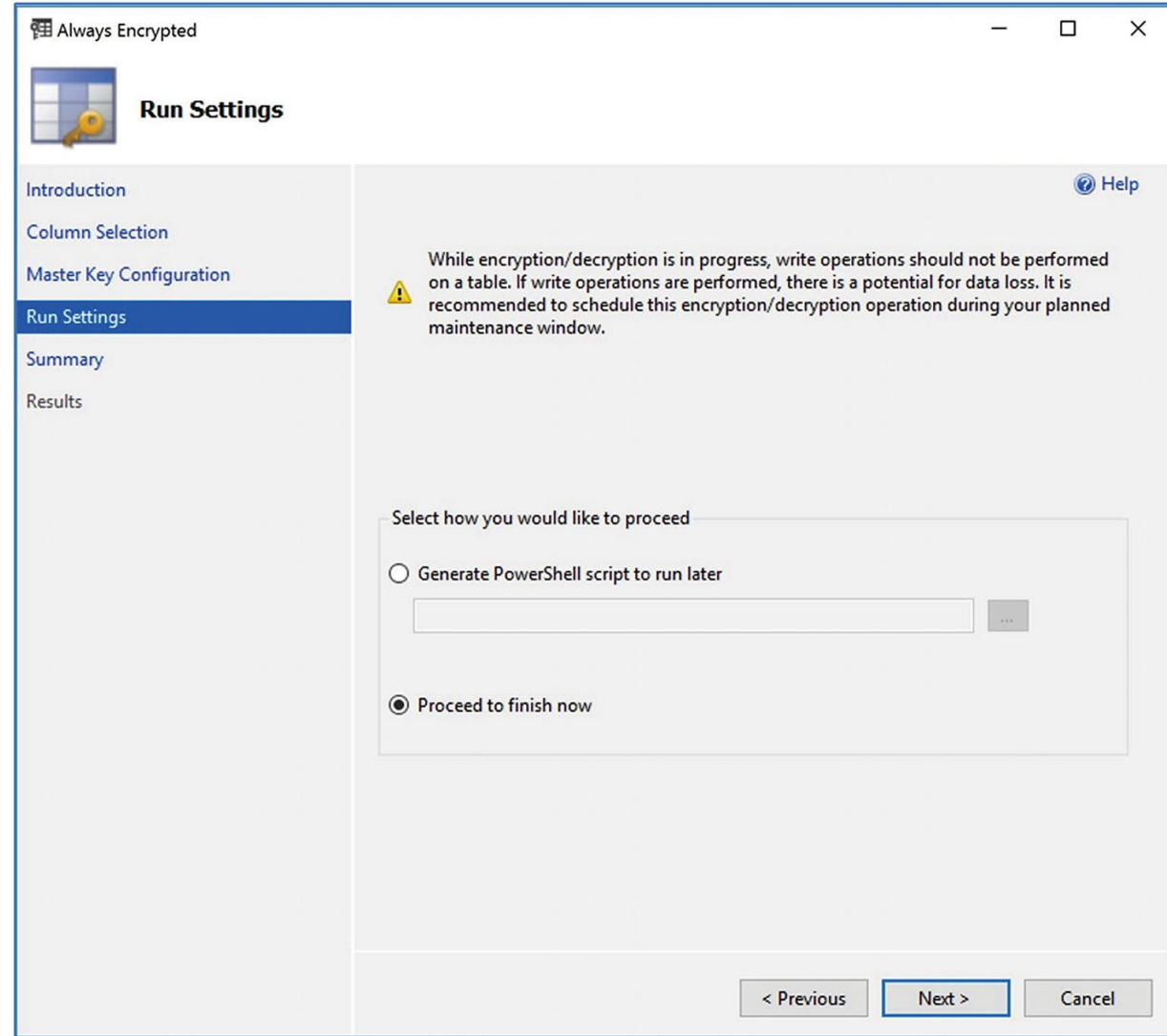


Figure 5-8 Run Settings page

Always Encrypted

- ▶ Limitations - not all features are supported by Always Encrypted.
- ▶ The following data types are not supported:
 - XML
 - TIMESTAMP
 - ROWVERSION
 - IMAGE
 - NTEXT
 - TEXT
 - SQL_VARIANT
 - HIERARCHYID
 - GEOGRAPHY
 - GEOMETRY

Always Encrypted

- User defined-types are not supported.
- FILESTREAM columns are not supported.
- Columns with the ROWGUIDCOL property specified are not supported.
- String columns are only supported when they use a BIN2 collation.
- Clustered and non-clustered and full-text index key columns are only supported for deterministic encryption.
- Columns referenced by computed columns are only supported when the expression does not perform unsupported operations.
- Sparse column sets are not supported.
- Columns that are referenced by statistics are not supported.
- Columns using alias data types are not supported.
- Partitioning key columns are not supported.
- Columns with default constraints are not supported.
- Columns referenced by unique constraints are only supported for deterministic encryption.
- Primary key columns are only supported when both of the following are true: Deterministic encryption is used; and
- Change tracking is not implemented on the column.
- Referencing columns in foreign keys are only supported when both of the following are true: Deterministic encryption is used; and
- The referenced and referencing columns are encrypted using the same key.
- Columns referenced by check constraints are not supported.
- Columns in tables that use change data capture are not supported.
- Columns that are masked using Dynamic Data Masking are not supported.



Summary

- ▶ Encryption is used to provide defence in depth.
- ▶ SQL Server uses an encryption hierarchy, which starts at the Service Master Key supported by DPAPI, then Database Master Keys then other keys and certificates.
- ▶ Data can be encrypted using passphrase or certificates.
- ▶ Keys and certificates are stored in external key vaults.
- ▶ TDE allows low overhead encryption of data at rest.
- ▶ Always Encrypted is a new technology that hides data from sysadmins.